PLONK Verifier 분석

개요

PLONK Verifier는 증명자가 생성한 영지식 증명을 검증하는 핵심 구성 요소입니다. 이 문서는 `plonk-core/src/proof_system/verifier.rs` 에 구현된 Verifier의 구조와 증명 검증 과정을 분석합니다.

Verifier 구조체 분석

주요 필드

```
pub struct Verifier<F, P, PC>
where
F: PrimeField,
P: TEModelParameters<BaseField = F>,
PC: HomomorphicCommitment<F>,
{
/// 특정 PLONK 회로를 검증하는 데 사용되는 VerificationKey
pub verifier_key: Option<PlonkVerifierKey<F, PC>>,


/// 회로 설명 (StandardComposer)
pub(crate) cs: StandardComposer<F, P>,
```

```
/// 전처리 단계에서 교환된 메시지를 저장
/// 증명을 검증할 때마다 복사되어 사용됨
pub preprocessed_transcript: Transcript,
}
```

역할과 기능

1. **VerifierKey**: 회로별 검증에 필요한 키 정보 (선택자 다항식 약정, 시그마 다항식 약정 등)
2. **StandardComposer**: 회로의 구조 정보 (증명자와 동일한 회로 구조 필요)
3. **Transcript**: Fiat-Shamir 변환을 위한 상호작용 기록 (증명자와 동일한 챌린지 생성)

VerifierKey 구조 분석

주요 구성 요소

```
pub struct VerifierKey<F, PC>
where
F: PrimeField,
PC: HomomorphicCommitment<F>,
{
/// 회로 크기 (2의 거듭제곱으로 패딩되지 않은 실제 크기)
pub(crate) n: usize,

/// 산술 게이트 검증 키
```

```
pub(crate) arithmetic: arithmetic::VerifierKey<F, PC>,

/// 범위 게이트 선택자 약정
pub(crate) range_selector_commitment: PC::Commitment,

/// 논리 게이트 선택자 약정
pub(crate) logic_selector_commitment: PC::Commitment,

/// 고정 기저 스칼라 곱셈 선택자 약정
pub(crate) fixed_group_add_selector_commitment: PC::Commitment,

/// 가변 기저 스칼라 곱셈 선택자 약정
pub(crate) variable_group_add_selector_commitment: PC::Commitment,

/// 순열 검사를 위한 VerifierKey
pub(crate) permutation: permutation::VerifierKey<PC::Commitment>,

/// 룩업 게이트를 위한 VerifierKey
pub(crate) lookup: lookup::VerifierKey<F, PC>,
}
```

산술 게이트 VerifierKey

```
pub struct VerifierKey<F, PC> {
pub q_m: PC::Commitment, // 곱셈 선택자
pub q_l: PC::Commitment, // 왼쪽 와이어 선택자
pub q_r: PC::Commitment, // 오른쪽 와이어 선택자
pub q_o: PC::Commitment, // 출력 와이어 선택자
pub q_4: PC::Commitment, // 네 번째 와이어 선택자
pub q_c: PC::Commitment, // 상수 선택자
pub q_hl: PC::Commitment, // 해시 왼쪽 선택자
pub q_hr: PC::Commitment, // 해시 오른쪽 선택자
pub q_h4: PC::Commitment, // 해시 네 번째 선택자
pub q_arith: PC::Commitment, // 산술 게이트 활성화 선택자
}
```

순열 VerifierKey

```
pub struct VerifierKey<PCC> {
pub left_sigma: PCC, // 왼쪽 와이어 순열 다항식 약정
pub right_sigma: PCC, // 오른쪽 와이어 순열 다항식 약정
pub out_sigma: PCC, // 출력 와이어 순열 다항식 약정
pub fourth_sigma: PCC, // 네 번째 와이어 순열 다항식 약정
}
```

룩업 VerifierKey

```
pub struct VerifierKey<F, PC> {
pub q_lookup: PC::Commitment, // 룩업 선택자 약정
pub table_1: PC::Commitment, // 룩업 테이블 1 약정
pub table_2: PC::Commitment, // 룩업 테이블 2 약정
pub table_3: PC::Commitment, // 룩업 테이블 3 약정
pub table_4: PC::Commitment, // 룩업 테이블 4 약정
}
```

증명 검증 과정 분석

1. 전처리 단계 (Preprocessing)

```
pub fn preprocess(&mut self, commit_key: &PC::CommitterKey) -> Result<(), Err
```

- `StandardComposer` 를 통해 VerifierKey 생성
- 회로의 선택자 다항식들에 대한 약정 생성
- 전처리된 transcript 저장

2. 주요 검증 단계

`verify` 메서드에서 `proof.verify()` 호출하여 수행되는 단계들:

2.1 도메인 설정 및 초기화

```
let domain = GeneralEvaluationDomain::<F>::new(plonk_verifier_key.n)?;
```

```
let mut transcript = self.preprocessed_transcript.clone();
```

- 증명자와 동일한 평가 도메인 생성
- 전처리된 transcript 복사하여 독립성 보장

2.2 공개 입력 처리

```
transcript.append(b"pi", pub_inputs);
```

- 공개 입력을 transcript에 추가
- 증명자와 동일한 공개 입력 공유 확인

2.3 증인 약정 처리

```
transcript.append(b"w_l", &self.a_comm);
transcript.append(b"w_r", &self.b_comm);
transcript.append(b"w_o", &self.c_comm);
transcript.append(b"w_4", &self.d_comm);
```

- 증명에 포함된 증인 다항식 약정들을 transcript에 추가
- 증명자와 동일한 순서로 처리하여 동일한 챌린지 생성

2.4 룩업 관련 처리

```
// 테이블 압축 챌린지 생성
let zeta = transcript.challenge_scalar(b"zeta");
transcript.append(b"zeta", &zeta);
```

```
// 룩업 다항식 약정들을 transcript에 추가
transcript.append(b"f", &self.f_comm);
transcript.append(b"h1", &self.h_1_comm);
transcript.append(b"h2", &self.h_2_comm);
```

2.5 순열 챌린지 생성

```
let beta = transcript.challenge_scalar(b"beta");
let gamma = transcript.challenge_scalar(b"gamma");
let delta = transcript.challenge_scalar(b"delta");
let epsilon = transcript.challenge_scalar(b"epsilon");

// 챌린지들이 서로 다른지 확인
assert!(beta != gamma, "challenges must be different");
// ... 다른 챌린지들도 동일하게 확인
```

보안 검증:

- 모든 챌린지가 서로 다른 값인지 확인
- 순열 논증의 건전성 보장

2.6 몫 다항식 관련 처리

```
// 몫 다항식 챌린지 생성
let alpha = transcript.challenge_scalar(b"alpha");
```

```
let range_sep_challenge = transcript.challenge_scalar(b"range separation challe

// ... 다른 분리 챌린지들도 생성


// 몫 다항식 약정들을 transcript에 추가

transcript.append(b"t_1", &self.t_1_comm);

transcript.append(b"t_2", &self.t_2_comm);

// ... t_8까지 모든 부분 추가
```

2.7 평가 점 챌린지 및 기본 계산

```
let z_challenge = transcript.challenge_scalar(b"z");


// 소멸 다항식 평가

let z_h_eval = domain.evaluate_vanishing_polynomial(z_challenge);


// 첫 번째 라그랑주 다항식 평가

let l1_eval = compute_first_lagrange_evaluation(&domain, &z_h_eval, &z_challeng
```

2.8 r0 계산 (핵심 검증 값)

```
let r0 = self.compute_r0(

&domain,

&pub_inputs.as_evals(domain.size()),
```

```
alpha, beta, gamma, delta, epsilon,

z_challenge, l1_eval,

self.evaluations.perm_evals.permutation_eval,

self.evaluations.lookup_evals.z2_next_eval,

self.evaluations.lookup_evals.h1_next_eval,

self.evaluations.lookup_evals.h2_eval,

lookup_sep_challenge,

);
```

r0의 역할:

- 모든 제약 조건의 만족을 하나의 값으로 통합
- 공개 입력, 순열, 룩업 제약 조건을 모두 포함

2.9 평가값들을 transcript에 추가

```
// 와이어 평가값들

transcript.append(b"a_eval", &self.evaluations.wire_evals.a_eval);

transcript.append(b"b_eval", &self.evaluations.wire_evals.b_eval);

transcript.append(b"c_eval", &self.evaluations.wire_evals.c_eval);

transcript.append(b"d_eval", &self.evaluations.wire_evals.d_eval);



// 순열 평가값들

transcript.append(b"left_sig_eval", &self.evaluations.perm_evals.left_sigma_eval

// ... 다른 평가값들도 추가
```

```
// 룩업 평가값들
transcript.append(b"f_eval", &self.evaluations.lookup_evals.f_eval);
// ... 다른 룩업 평가값들도 추가
```

2.10 선형화 약정 계산

```
let lin_comm = self.compute_linearisation_commitment::<P>(
&domain, alpha, beta, gamma, delta, epsilon, zeta,
range_sep_challenge, logic_sep_challenge,
fixed_base_sep_challenge, var_base_sep_challenge,
lookup_sep_challenge, z_challenge, l1_eval,
plonk_verifier_key,
);
```

선형화의 목적:

- 검증자가 효율적으로 검증할 수 있도록 다항식을 선형 결합으로 재구성
- 증명자가 제공한 평가값들을 사용하여 약정 재계산

2.11 테이블 약정 재구성

```
let zeta_sq = zeta.square();
let table_comm = PC::multi_scalar_mul(
&[
plonk_verifier_key.lookup.table_1.clone(),
```

```
plonk_verifier_key.lookup.table_2.clone(),

plonk_verifier_key.lookup.table_3.clone(),

plonk_verifier_key.lookup.table_4.clone(),

],

&[F::one(), zeta, zeta_sq, zeta_sq * zeta],

);
```

2.12 배치 열기 검증

```
// 집계 증인 (Aggregate Witness) 검증

let  aw_challenge: F = transcript.challenge_scalar(b"aggregate_witness");


let  aw_commits = [

lin_comm, // 선형화 약정

plonk_verifier_key.permutation.left_sigma, // 시그마 약정들

// ... 다른 약정들

self.a_comm, self.b_comm, self.c_comm, self.d_comm, // 증인 약정들

];


let  aw_evals = [

-r0, // 핵심 검증 값

self.evaluations.perm_evals.left_sigma_eval,

// ... 다른 평가값들

];
```

```
// 첫 번째 배치 검증
PC::check(verifier_key, &aw_commits, &z_challenge, aw_evals, &self.aw_opening, a


// 시프트된 집계 증인 (Shifted Aggregate Witness) 검증
let  saw_challenge: F = transcript.challenge_scalar(b"aggregate_witness");


let  saw_commits = [
self.z_comm, // 순열 다항식 약정
self.a_comm, self.b_comm, self.d_comm, // 시프트된 증인 약정들
self.h_1_comm, self.z_2_comm, // 룩업 관련 약정들
table_comm,
];


let  saw_evals = [
self.evaluations.perm_evals.permutation_eval,
self.evaluations.custom_evals.get("a_next_eval"), // 시프트된 평가값들
// ... 다른 시프트된 평가값들
];


// 두 번째 배치 검증
```

```
PC::check(verifier_key, &saw_commits, &(z_challenge * domain.element(1)), saw_ev
```

배치 검증의 효율성:

- 두 개의 배치 검증으로 모든 다항식 열기 확인
- 첫 번째: z_challenge에서의 평가값들 검증
- 두 번째: z_challenge * ω (시프트된 점)에서의 평가값들 검증

핵심 검증 알고리즘

compute_r0 함수 분석

```
fn  compute_r0(&self, domain, pub_inputs, alpha, beta, gamma, delta, epsilon, z_
```

r0 계산 공식:

```
r0 = pi_eval - b - c - d - e
```

여기서:

- `pi_eval` : 공개 입력 다항식의 z_challenge에서의 평가
- `b` : 순열 제약 조건 항
- `c` : 첫 번째 라그랑주 다항식 항 (l1_eval * alpha^2)
- `d` : 룩업 제약 조건 항
- `e` : 룩업 초기화 항

compute_linearisation_commitment 함수 분석

선형화 약정은 다음 구성 요소들의 선형 결합:

1. **산술 게이트 기여분**: 산술 제약 조건
2. **범위 게이트 기여분**: 범위 제약 조건
3. **논리 게이트 기여분**: 논리 제약 조건
4. **타원 곡선 게이트 기여분**: 고정/가변 기저 스칼라 곱셈
5. **룩업 게이트 기여분**: 룩업 제약 조건
6. **순열 기여분**: 순열 제약 조건
7. **몫 다항식 기여분**: 8개 부분의 몫 다항식

보조 메서드 분석

1. `compute_first_lagrange_evaluation` - 첫 번째 라그랑주 다항식 평가

```
pub fn compute_first_lagrange_evaluation<F>(
domain: &GeneralEvaluationDomain<F>,
z_h_eval: &F,
z_challenge: &F,
) -> F
```

수학적 공식:

```
L_0(X) = (X^n - 1) / (n * (X - 1))
```

2. `compute_barycentric_eval` - 중심좌표 평가

공개 입력 다항식을 효율적으로 평가하는 방법:

- 0이 아닌 평가값들만 처리
- 배치 역원 계산 사용

성능 및 보안 고려사항

성능 최적화

1. **배치 검증**: 두 개의 배치로 모든 다항식 열기 검증
2. **선형화**: 다항식 약정들을 선형 결합으로 효율적 처리
3. **중심좌표 평가**: 공개 입력 다항식의 효율적 평가

보안 특성

1. **챌린지 독립성**: 모든 챌린지가 서로 다른 값인지 확인
2. **Fiat-Shamir 일관성**: 증명자와 동일한 transcript 처리
3. **완전성**: 올바른 증명은 항상 검증 통과
4. **건전성**: 잘못된 증명은 높은 확률로 거부

요구사항 충족 확인

- **요구사항 3.2**: Verifier의 구조와 역할 파악 ✓
- **요구사항 3.4**: 증명 검증 과정의 각 단계 분석 ✓

이 분석을 통해 PLONK Verifier가 어떻게 증명자가 생성한 영지식 증명을 효율적이고 안전하게 검증하는지, 그리고 각 단계에서 어떤 암호학적 검증이 수행되는지 이해할 수 있습니다.